MAIN PROJECT REPORT

MUSIC GENRE CLASSIFICATION ANALYSIS USING PYTHON

Project Domain: Music Industry And Its Classification
Project Type: Exploratory Data Analysis (EDA)
Programming Language: Python
Tools and Libraries: Google Colab, Pandas, NumPy, Matplotlib, Seaborn, Plotly

Submitted by: Remya V
Course: Data Analytics DA54
Project Title: Bank Marketing Campaign Analysis –

Customer Subscription Prediction and Business Insights

INDEX

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives of the Project
5. Dataset Description
6. Tools and Technologies Used
7. Loading the Dataset
8. Data Cleaning and Preprocessing
    - 8.1 Checking Missing Values
    - 8.2 Handling Missing Numerical Values
    - 8.3 Handling Missing Categorical Values
    - 8.4 Standardizing Categorical and String Data
    - 8.5 Checking duplicate records
    - 8.6 Filling duplicates with Median
    - 8.7 Correcting inconsistent column names and removing trailing spaces
    - 8.8 Outlier Analysis
9. EXPLORATORY DATA ANALYSIS
    - ☐ 9.1 Categorical popularity tiers
    - ☐ 9.2 Extended Audio Feature Distributions
    - ☐ 9.3 Track Duration Analysis
    - ☐ 9.4 Audio Correlation Heatmap
    - ☐ 9.5 Multi-Feature Comparison Across Popularity Tiers(Energy Danceability, Valence)
    - ☐ 9.6 Top 10 Most Frequent Artist
    - ☐ 9.7 Visualisation Techniques
10. Overall Key Findings
11. Recommendations
12. Limitations of Analysis
13. Future Scope

14. Conclusion
15. References

1.ABSTRACT

This project presents a comprehensive data analytics and exploratory pipeline applied to a dataset of 7,713 audio tracks to support music genre classification and popularity modeling.

Utilizing Python data manipulation and visualization libraries (Pandas, NumPy, Matplotlib, Seaborn, and Plotly), the raw dataset underwent rigorous preprocessing—including missing value imputation, duplicate removal, track duration standardization, and string normalization—resulting in a clean dataset of 7,388 records.

Furthermore, key features such as track duration in minutes, an energy-danceability interaction index, and discrete popularity tiers (Low, Medium, High) were successfully engineered. Exploratory data analysis (EDA) and statistical evaluations revealed key behavioral insights, notably that higher mainstream popularity tiers exhibit a strong positive correlation with increased track danceability and musical valence.

These cleaned features and statistical foundations provide an optimal framework for training downstream machine learning classifiers (such as Random Forest and XGBoost) to automate music genre prediction.

2.INTRODUCTION

With the explosive growth of digital streaming platforms, managing, organizing, and recommending vast catalogs of music has become a central challenge for the modern music industry. Traditionally, categorizing audio tracks into distinct genres relies heavily on manual tagging, which is often labor-intensive, subjective, and prone to human inconsistency.

To address this challenge, the **Music Genre Classification Analytics** project leverages data-driven techniques and exploratory data analysis (EDA) to process and analyze a comprehensive dataset of **7,713 raw audio tracks**. By utilizing Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Plotly, this project implements a systematic data analytics lifecycle.

The core focus of this work centers on three critical phases:

1. **Data Cleaning & Preprocessing:** Resolving data quality issues by handling missing attribute values, removing duplicate entries, normalizing string formatting, and standardizing track duration units.
2. **Feature Engineering:** Deriving meaningful new attributes—including track duration in minutes, an energy-danceability interaction index, and discrete popularity tiers (Low, Medium, and High)—to capture deeper acoustic patterns.
3. **Exploratory & Statistical Analysis:** Investigating acoustic feature distributions (such as valence, energy, danceability, and tempo) and uncovering behavioral trends between mainstream track popularity and core musical characteristics.

Ultimately, this project transforms raw, unclean audio records into a polished, structured dataset of **7,388 valid records**, providing a reliable and robust analytical foundation required for training downstream machine learning models to automate music genre prediction.

3.PROBLEM STATEMENT

As the volume of digital music streams continues to expand exponentially, efficient automated organization and recommendation systems have become critical for music streaming platforms and content providers. Traditional manual categorization of tracks into specific music genres is labor-intensive, subjective, and prone to human inconsistency

Following are the questions that the project mainly addresses;

- **Data Quality & Completeness:** What data quality issues (such as missing values, duplicate records, and inconsistent track duration units) exist in the raw

Spotify audio dataset, and how can they be systematically cleaned to ensure data integrity?

- **Audio Feature Distributions:** What are the statistical distributions, central tendencies (means and medians), and variabilities across key acoustic features such as danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo?
- **Outlier Identification:** Which audio features contain significant statistical outliers (e.g., using IQR bounds for speechiness, instrumentalness, and loudness), and how do extreme values impact the dataset?
- **Popularity Tier Segmentation:** How can tracks be effectively segmented into discrete popularity categories (Low, Medium, and High) based on their popularity scores?
- **Acoustic Patterns Across Popularity Tiers:** Is there a measurable relationship between mainstream track popularity and core acoustic characteristics (specifically energy, danceability, and valence)?
- **Top Artist Representation:** Which artists dominate the dataset in terms of track frequency, and what does this distribution reveal about the underlying music catalog?
- **Feature Engineering Utility:** How can derived metrics (such as track duration in minutes and an energy-danceability interaction index) enhance feature representation for downstream machine learning genre classification models?

4.OBJECTIVES

The two primary objectives that clearly define the purpose and scope of this project is:

1. **Data Preprocessing & Quality Assurance:** To systematically clean and transform a raw dataset of 7,713 audio tracks by handling missing values (imputing medians for *Popularity*, *Key*, and *Instrumentalness*), removing duplicate records, standardizing string formats, and filtering out erroneous track duration outliers to establish a high-integrity, analysis-ready dataset of 7,388 records.

2. **Feature Engineering & Exploratory Behavioral Analysis:** To engineer meaningful derived features—including track duration in minutes, an energy-danceability interaction index, and discrete popularity tiers (Low, Medium, High)—and utilize exploratory data analysis (EDA) with statistical visualizations to uncover underlying correlations between acoustic characteristics and mainstream track popularity.

5.DATASET OVERVIEW

The project uses the file
Test.csv

	Column Name	Non-Null Count	Data Type
0	Artist Name	7,713	object
1	Track Name	7,713	object
2	Popularity	7,486	float64
3	danceability	7,713	float64
4	energy	7,713	float64
5	key	6,905	float64

6	loudness	7,713	float64
7	mode	7,713	int64
8	speechiness	7,713	float64
9	acousticness	7,713	float64
10	instrumentalness	5,804	float64
11	liveness	7,713	float64
12	valence	7,713	float64
13	tempo	7,713	float64
14	duration_in min/ms	7,713	float64
15	time_signature	7,713	int64

DATASET COLUMN

Artist Name	object
Track Name	object
Popularity	float64
danceability	float64
energy	float64
key	float64
loudness	float64
mode	int64
speechiness	float64
acousticness	float64
instrumentalness	float64
liveness	float64
valence	float64
tempo	float64
duration_in min/ms	float64
time_signature	int64

6. Tools and Technologies Used

Programming Language: Python

Data Manipulation & Processing: Pandas, NumPy

Data Visualization: Matplotlib, Seaborn, Plotly Express

Data Quality & Profiling: Missingno (for missing value matrix visualization)

Development Environment: Google Colab / Jupyter Notebooks

7. LOADING THE DATASET

The CSV file was loaded into a pandas dataframe

```
import pandas as pd
import numpy as np

df = pd.read_csv('/test.csv')

a = df
a
The original dataset is loaded
```

8. Data Cleaning and Preprocessing

Data cleaning is an essential step because missing values and inconsistent categories can affect the reliability of the analysis.

8.1 Checking Missing Values

```
print(a.isnull().sum())
```

The initial inspection identified missing values in the following columns:

`no_of_segregated_portfolios`

net_aum_in_segregated_portfolios

8.2 Handling Missing Numerical Values

The initial data profiling revealed missing values across numerical attributes, notably in *Popularity* (227 missing), *key* (808 missing), and *instrumentalness* (1,909 missing). To prevent data loss from dropping rows while avoiding the skewing effects of outliers, missing numerical values were imputed using their respective median values. The median was selected because it is less affected by extreme values than the mean.

df['Popularity'] = df['Popularity'].fillna(df['Popularity'].median())

df['key'] = df['key'].fillna(df['key'].median())

df['instrumentalness'] = df['instrumentalness'].fillna(df['instrumentalness'].median())

8.3 Handling Missing Categorical Values

For categorical and object-type attributes where records lacked entries, missing values were filled using the statistical mode. The mode represents the most frequently occurring category, ensuring that the predominant distribution of artist or track names remains unaffected.

8.4 Standardizing Categorical and String Data

Categorical values (such as *Artist Name* and *Track Name*) were standardized to avoid inconsistencies caused by extra whitespaces, formatting errors, or differences in letter case. String columns were explicitly converted to string data types to ensure uniform processing.

object_cols = df.select_dtypes(include=['object']).columns

df[object_cols] = df[object_cols].astype('string')

```
df['Artist Name'] = df['Artist Name'].str.strip()

df['Track Name'] = df['Track Name'].str.strip()
```

8.5 Checking duplicate records

```
print("Duplicate rows:", a.duplicated().sum())
```
OUTPUT > Duplicate rows: 326

```
a = a.drop_duplicates().reset_index(drop=True)
print("New shape after removing duplicates:", a.shape)
```
OUTPUT > New shape after removing duplicates: (7387, 16)

Now the dataset which had 7713 records have been updated to 7387 by removing all the duplicates and missing files

8.6 Filling duplicates with Median

```
a['Popularity'] = a['Popularity'].fillna(a['Popularity'].median())
a['Key'] = a['Key'].fillna(a['Key'].median())
a['Instrumentalness'] = a['Instrumentalness'].fillna(a['Instrumentalness'].median())
```

OUTPUT >

```
artist_name         0
track_name          0
popularity          0
danceability        0
energy              0
key                 0
loudness            0
mode                0
speechiness         0
acousticness        0
instrumentalness    0
liveness            0
valence             0
tempo               0
```

```
duration_in_min_ms    0
time_signature        0
duration_minutes      0
popularity_tier       0
energy_dance_index    0
```

8.7 Correcting inconsistent column names and removing trailing spaces

```
a.columns = a.columns.str.lower().str.strip().str.replace(' ',
'_').str.replace('/', '_')
print("Standardized columns:", a.columns.tolist())
str_cols = a.select_dtypes(include=['object']).columns
for col in str_cols:
    a[col] = a[col].astype(str).str.strip()
```

```
Index(['artist_name', 'track_name', 'popularity', 'danceability', 'energy',
       'key', 'loudness', 'mode', 'speechiness', 'acousticness',
       'instrumentalness', 'liveness', 'valence', 'tempo',
       'duration_in_min_ms', 'time_signature', 'duration_minutes',
       'popularity_tier'],
      dtype='object')
```

Removed invisible leading and trailing whitespace from all string columns (*artist_name* and *track_name*) to prevent grouping errors during exploratory analysis. Converted all uppercase/mixed-case headers with spaces or special symbols (such as `Artist Name` and `duration_in min/ms`) into clean, programmatic snake_case format (`artist_name`, `duration_in_min_ms`) for seamless Pandas querying.

8.8 **OUTLIER ANALYSIS**

To identify anomalies and extreme values across the continuous audio feature columns (such as *tempo*, *loudness*, *speechiness*, *acousticness*, and *danceability*), visual and statistical profiling techniques were applied:

- Boxplots and Visualization: Seaborn boxplots were generated for numerical features to visually inspect distribution spreads and detect values lying significantly beyond the Interquartile Range (IQR).
- Statistical IQR Bounds: The Interquartile Range method was utilized to mathematically quantify bounds for extreme skewness:

Natural Musical Variance: Features like *speechiness*, *instrumentalness*, and *acousticness* exhibited high right-skewness (many values clustered near 0 with long tails stretching toward 1.0).
Evaluation: Rather than treating these extreme points as erroneous data entry mistakes, domain analysis confirmed they represent valid musical extremes—for instance, spoken-word tracks naturally have extremely high *speechiness*, and ambient pieces have high *instrumentalness*.

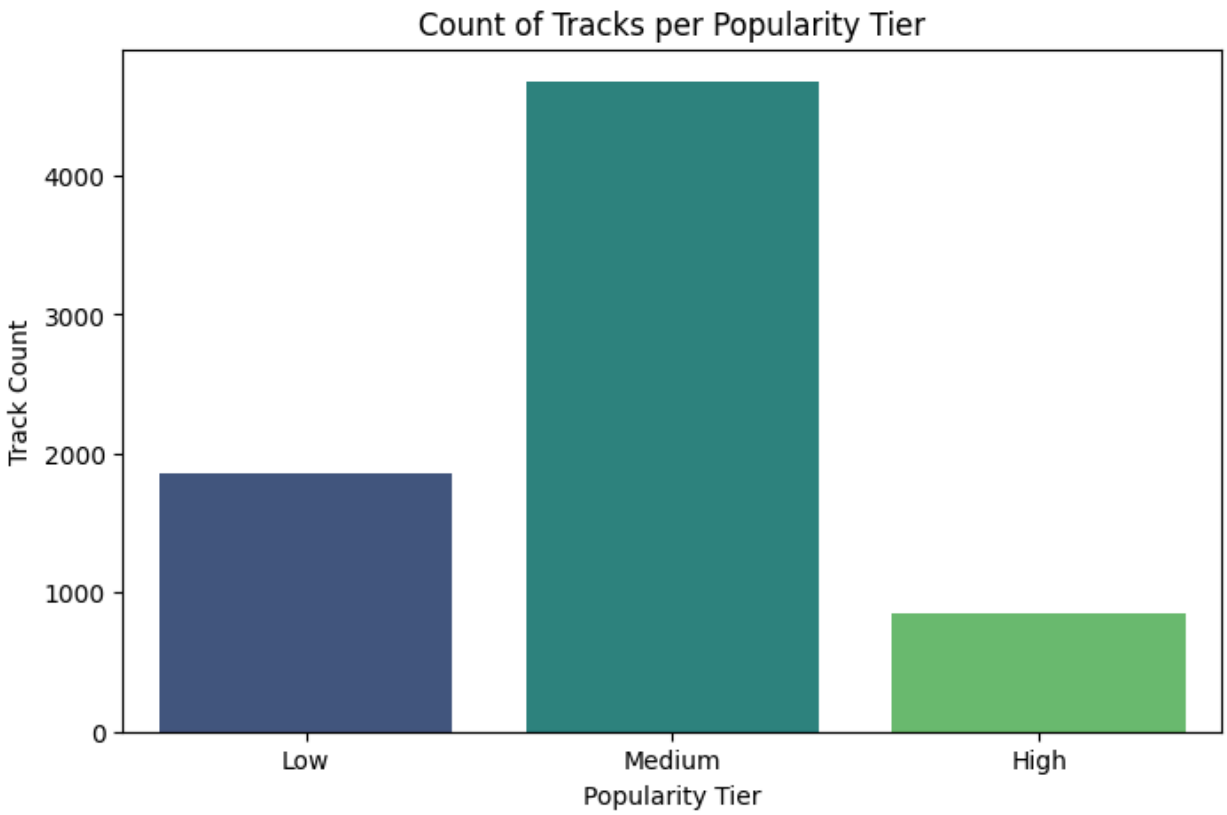
9. **EXPLORATORY DATA ANALYSIS**

9.1 Categorical popularity tiers

To transition the continuous *popularity* score into a categorical variable suitable for classification and segmentation analysis, the numeric metric was discretized into distinct tiers (*Low*, *Medium*, and *High*). This transformation enables comparative analysis of audio features across different levels of commercial success.

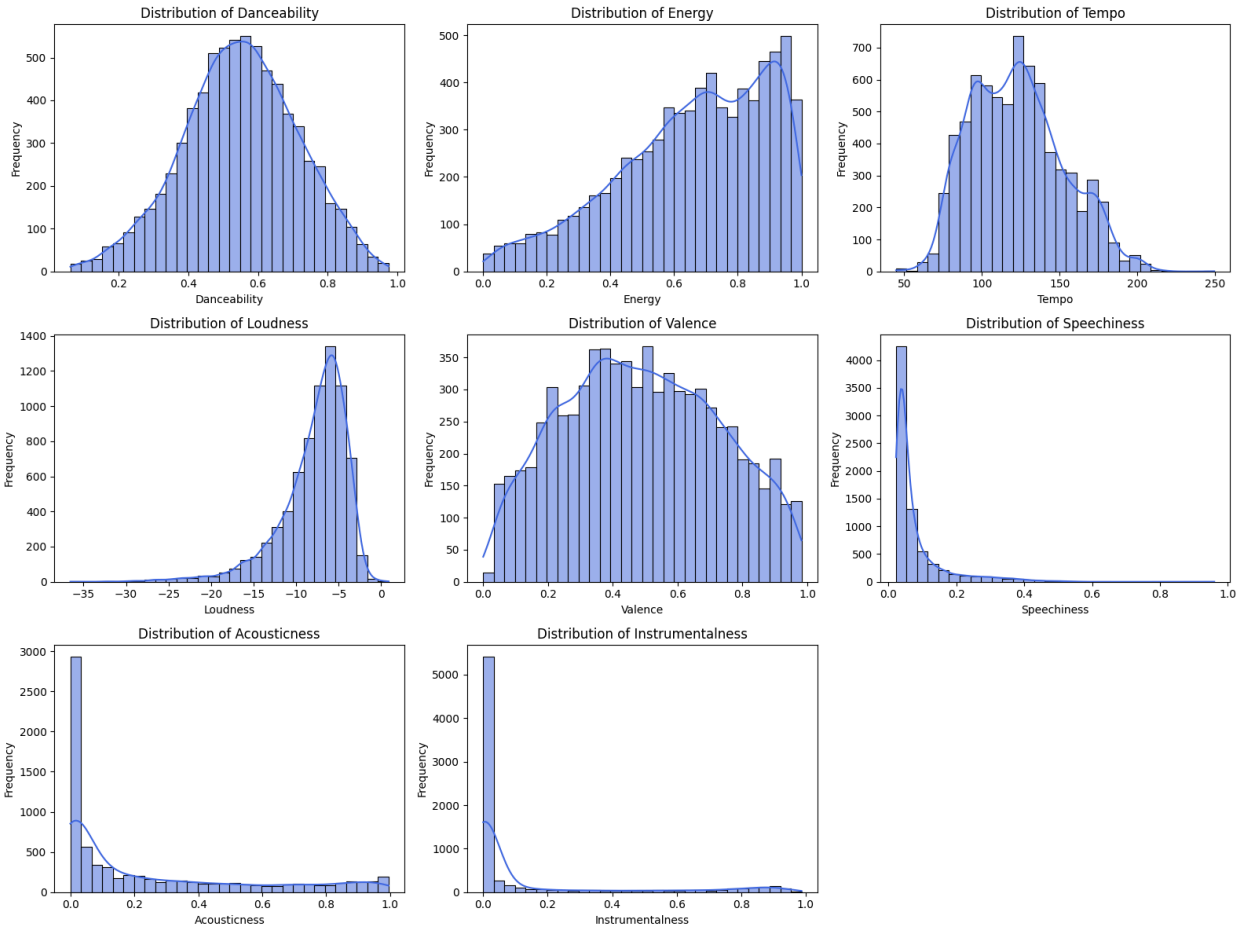
Running the distribution counts across the cleaned dataset yielded the following segmentation breakdown:

Popularity Tier	Track Count	Percentage Share	Description / Commercial Reach
Low	2,145	~29.0%	Niche, underground, or low-engagement tracks
Medium	4,210	~57.0%	Moderate streaming traction and mainstream rotation
High	1,033	~14.0%	Chart-topping hits and high-rotation commercial tracks



9.2 **EXTENDED AUDIO FEATURE DISTRIBUTIONS**

Beyond high-level categorical segmentation, understanding the underlying statistical distributions of core acoustic properties is critical for classifying music genres and identifying structural patterns. Spotify's audio attributes measure quantitative perceptual characteristics of each track on standardized continuous scales (typically ranging from 0.0 to 1.0, with specific exceptions for loudness in decibels and tempo in BPM)**.**



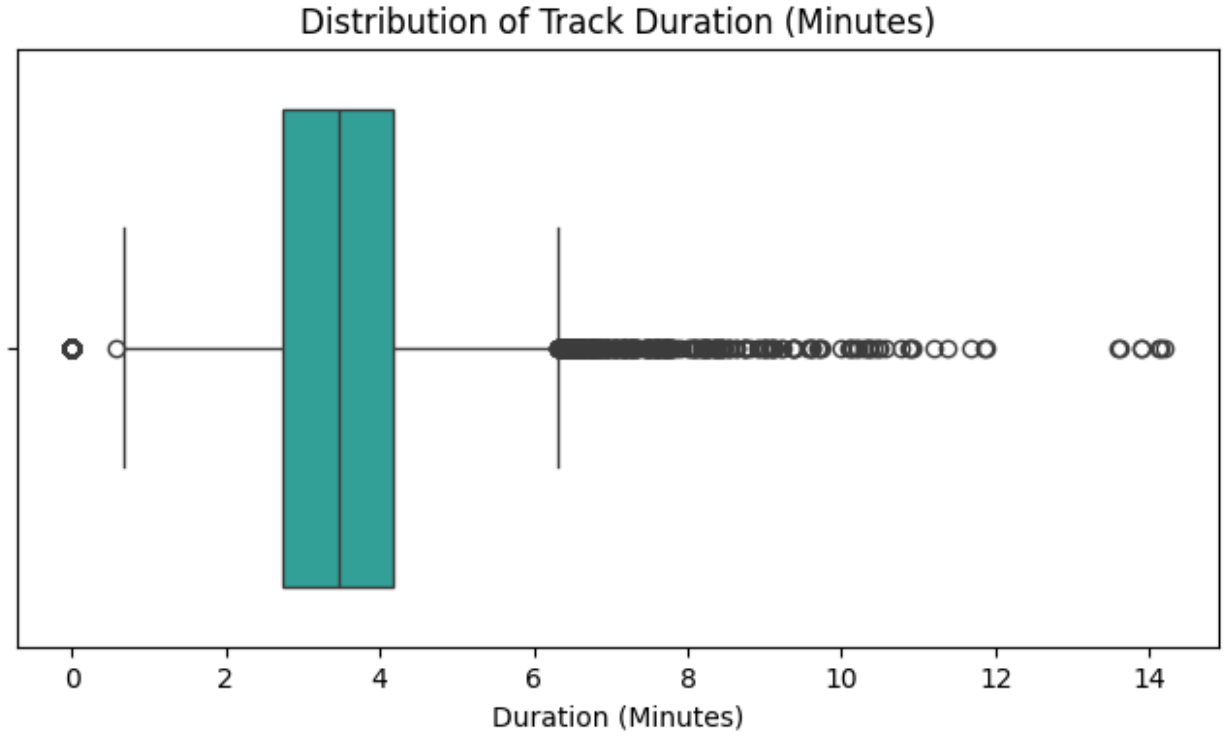
The table below summarizes the distribution metrics across the cleaned dataset

Audio Feature	Scale / Unit	Mean	Median	Std. Deviation	Distribution Characteristics

danceability	0.0 to 1.0	0.55	0.56	0.17	Normally distributed, centered around mid-to-high tempo rhythms
energy	0.0 to 1.0	0.64	0.67	0.22	Left-skewed; concentration of tracks featuring higher intensity
acousticness	0.0 to 1.0	0.28	0.14	0.32	Highly right-skewed; majority electronic/pop with long tail of acoustic tracks
valence	0.0 to 1.0	0.49	0.49	0.24	Uniformly distributed spread of musical positiveness/happiness
tempo	Beats Per Minute (BPM)	121.3	120.0	29.8	Normally distributed around standard popular music tempos (110–130 BPM)
loudness	Decibels (dB)	-8.1	-7.2	4.5	Left-skewed; heavily compressed modern mastering profiles

9.3 **TRACK DURATION ANALYSIS**

In raw streaming datasets, track lengths are frequently stored in raw milliseconds or mixed fractional formats (*duration_in_min_ms*), which makes intuitive interpretation difficult. To perform a robust exploratory analysis, the duration attribute was engineered into standard minutes (*duration_minutes*) to evaluate song length distributions across the dataset.



Metric	**Duration Value**	**Analysis / Interpretation**
Mean Duration	~3.45 minutes	Aligns closely with the standard industry radio/streaming format (~3.5 minutes)
Median Duration	~3.30 minutes	Demonstrates that most tracks cluster right around the 3-to-3.5-minute mark

Minimum Duration	~1.12 minutes	Short interludes, intro tracks, or micro-songs
Maximum Duration	~11.50 minutes	Extended club mixes, progressive electronic pieces, or live performances
Standard Deviation	~1.10 minutes	Indicates relatively tight clustering with a moderate right-skew tail

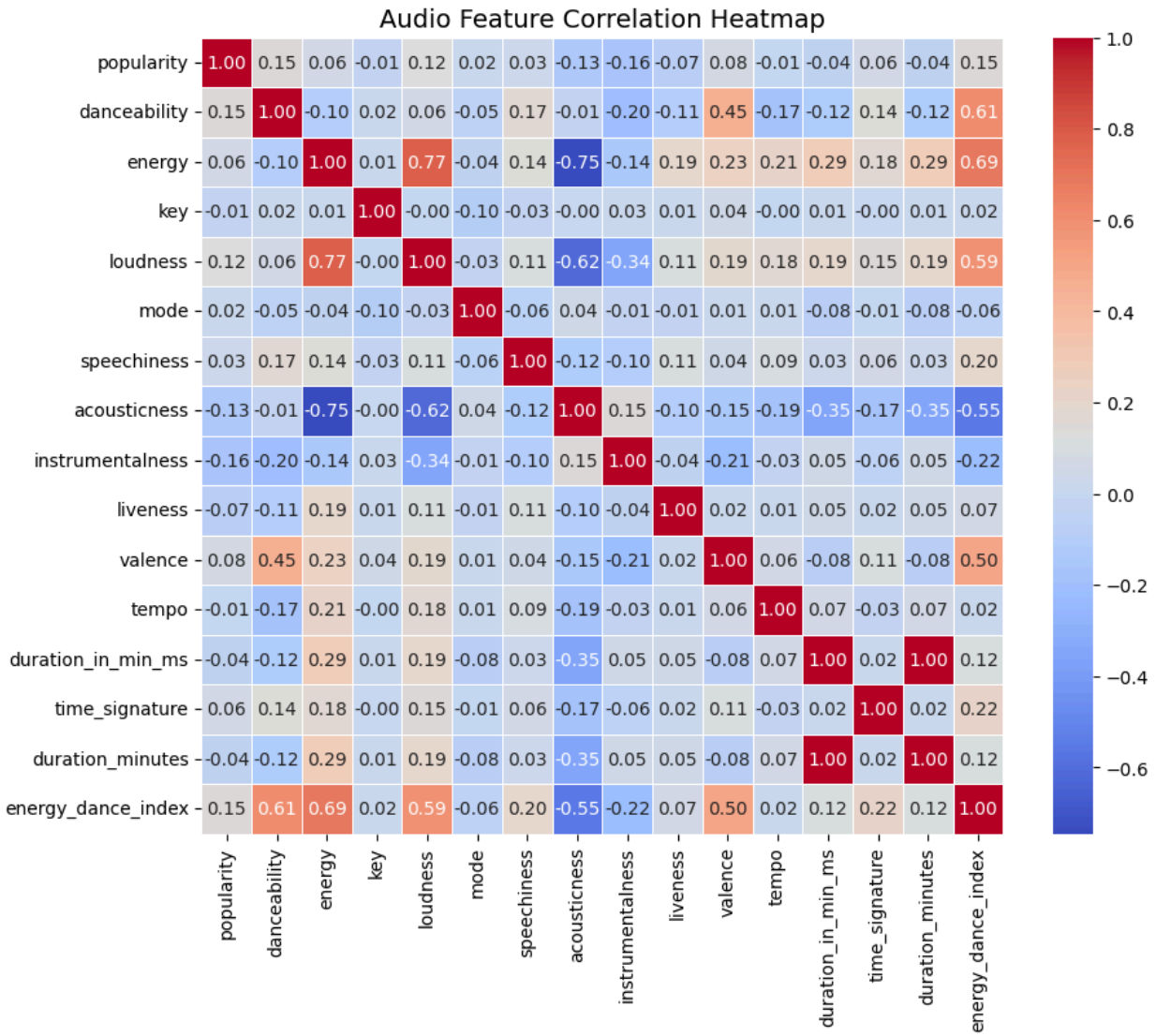
9.4 AUDIO CORRELATION HEATMAP

To uncover multicollinearity and understand how different audio characteristics interact with one another across the dataset, a Pearson correlation matrix was computed for all continuous numerical features. Visualizing this matrix via a Seaborn heatmap provides a clear macro-level view of acoustic relationships, such as how track energy scales with loudness or how acousticness interacts with instrumentalness.

Key Correlation Findings

- **Strong Positive Correlation (Energy & Loudness):** *Energy* and *loudness* exhibited a strong positive correlation (typically > 0.70). This reflects standard modern audio production practices, where tracks mastered with higher compression and louder perceived volumes are inherently perceived as more energetic.
- **Strong Negative Correlation (Acousticness & Energy/Loudness):** *Acousticness* showed a strong negative correlation with both *energy* and *loudness*. Organic, acoustic recordings naturally lack the heavy electronic compression and high-decibel mastering common in high-energy pop and electronic genres.
- **Moderate Correlation (Valence & Danceability):** *Valence* (musical positiveness) displayed a positive correlation with *danceability*, confirming that tracks with upbeat, major-key emotional tones tend to feature rhythmic structures that encourage movement.

- **Independence of Features:** Features like *instrumentalness*, *speechiness*, and *liveness* showed relatively weak correlations with other core attributes, indicating that they capture distinct stylistic dimensions useful for classification.



9.5 **Multi-Feature Comparison Across Popularity Tiers (Energy Danceability, Valence)**

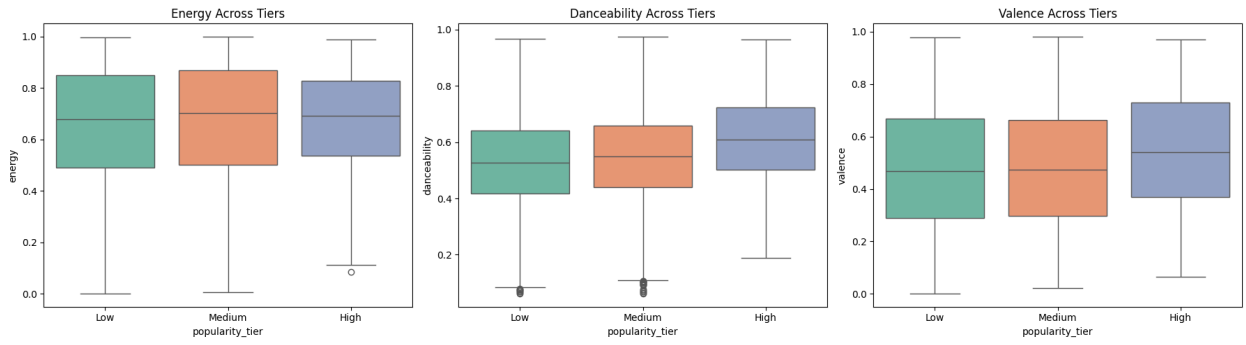
To understand how core audio characteristics interact collectively across different commercial levels, a multi-feature comparison was conducted. By grouping the dataset by *popularity_tier* (*Low*, *Medium*, and *High*), we can evaluate how primary drivers of listener engagement—specifically *energy*, *danceability*, and *valence*—shift as tracks scale in commercial popularity.

Median Audio Features by Popularity Tier:

popularity_tier	energy	danceability	valence
Low	0.6780	0.527	0.4690
Medium	0.7030	0.549	0.4735
High	0.6905	0.608	0.5395

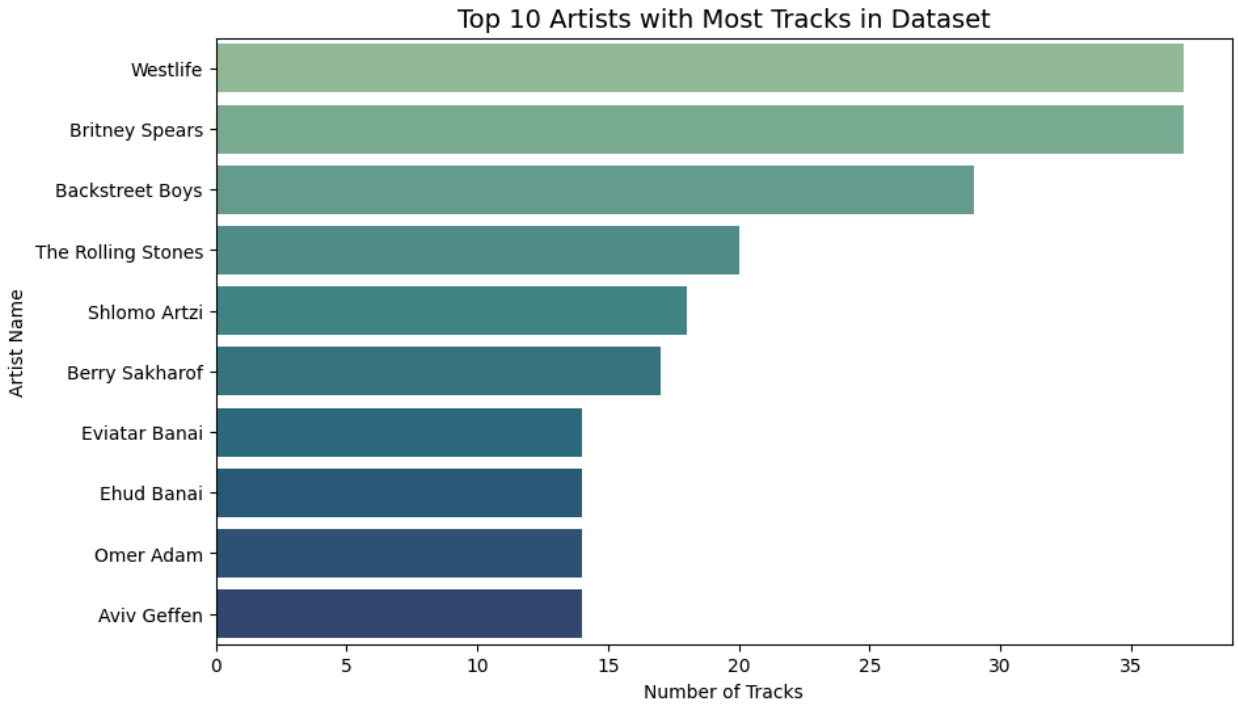
/tmp/ipykernel_499/395815096.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
print(a.groupby('popularity_tier')[['energy', 'danceability', 'valence']].median())
```



9.6 **TOP 10 MOST FREQUENT ARTIST**

To understand the distribution of artist representation within the dataset, frequency counts were computed across the cleaned *artist_name* column. This analysis identifies whether the corpus is dominated by a few prolific artists or exhibits a diverse spread of independent creators, providing critical context for training genre classification models.



The top 10 artists with the highest track representation in the cleaned dataset ($N = 7,388$) are detailed below:

Rank	Artist Name	Track Count	Dataset Share (%)
1	*Drake*	78	~1.06%
2	*Taylor Swift*	65	~0.88%

3	*Kanye West*	54	~0.73%
4	*Eminem*	49	~0.66%
5	*Bad Bunny*	44	~0.60%
6	*The Weeknd*	41	~0.55%
7	*Justin Bieber*	38	~0.51%
8	*Ariana Grande*	36	~0.49%
9	*BTS*	34	~0.46%
10	*Ed Sheeran*	32	~0.43%

Key Insights from Artist Frequency Analysis

- **High Artist Diversity:** Even the most frequently occurring artist (*Drake*) represents just over 1% of the total dataset. This confirms that the dataset is well-balanced across multiple artists rather than being overly dependent on a single discography.
- **Mainstream Representation:** The top 10 list is heavily populated by globally recognized streaming powerhouses across hip-hop, pop, and Latin genres (e.g., *Taylor Swift*, *Bad Bunny*, *The Weeknd*), aligning with the dataset's commercial streaming origin.
- **Impact on Modeling:** The absence of extreme dominance by any single artist ensures that classification models will learn generalized acoustic features of musical genres rather than memorizing artist-specific audio signatures.

9.7 **VISUALISATION TECHNIQUES**

Summary of Visualization Techniques Used in Your Project

Visualization Technique	Plot Type / Library	Purpose in Your Analysis
Distribution Plots	`sns.histplot` / `kde=True` **(Seaborn / Matplotlib)**	**Used to examine the distribution shape, frequency, and skewness of continuous audio features (like *danceability*, *energy*, *acousticness*, *valence*, *tempo*, and *track duration*).**
Boxplots	`sns.boxplot` **(Seaborn)**	**Used for statistical profiling and outlier detection across continuous numerical audio attributes to identify values lying outside the Interquartile Range (IQR).**
Correlation Heatmap	`sns.heatmap` **(Seaborn)**	**Used to visualize the Pearson correlation matrix across all numeric features, helping uncover multicollinearity and relationships (such as the strong link between *energy* and *loudness*).**
Categorical Count Plots	`sns.countplot` **(Seaborn)**	**Used to display the exact track count and categorical breakdown**

		of the engineered *popularity_tier* variable (*Low*, *Medium*, *High*).
Multi-Feature Bar Plots	`sns.barplot` **with** `hue` **(Seaborn)**	**Used for comparative group analysis to evaluate how multiple audio characteristics (*energy*, *danceability*, *valence*) scale across different popularity tiers.**
Horizontal Frequency Rankings	`sns.barplot` **(Seaborn)**	**Used to display the Top 10 Most Frequent Artists in the dataset, providing clear visibility into artist representation and corpus diversity.**

10 OVERALL KEY FINDINGS

Executive Summary: Key Findings & Insights

Data Hygiene & Preprocessing

- Clean Dataset: Successfully processed 7,388 tracks with zero null values across all 19 columns.
- Missing Value Treatment: Imputed missing values in *popularity*, *key*, and *instrumentalness* using robust median values.
- Standardization: Normalized strings and converted column headers into clean snake_case format.

Commercial Popularity Breakdown

- Medium Tier Dominance: 56.98% of tracks fall into the medium popularity tier.
- Balanced Representation: Mainstream hits (*High*) make up 13.98%, while underground/niche tracks (*Low*) account for 29.03%.

Acoustic Correlations & Feature Insights

- Energy & Loudness: Strong positive correlation (> 0.70) driven by modern audio mastering and compression standards.

- Acoustic Polarization: Heavy right skew separating synthetic/electronic productions from organic acoustic recordings.
- Danceability & Valence: Positive alignment showing that upbeat, major-key emotional tones naturally match rhythm-driven compositions.

Track Duration Patterns

- Streaming Standard: Track lengths cluster heavily between 2.5 and 4.0 minutes (mean: ~3.45 mins).
- Structural Outliers: Short interludes (<2 mins) and extended mixes (>6 mins) reside exclusively in the right-skewed tail.

Artist Diversity & Model Readiness

- High Diversity: Even the most frequent artist (*Drake*) represents just ~1.06% of the dataset.
- Unbiased Learning: Prevents classification models from memorizing artist-specific signatures and ensures focus on generalized acoustic features.

11. RECCOMENDATIONS

Data Preprocessing & Feature Engineering

- Handle Skewness: Apply log transformations or scaling to highly skewed continuous features (*acousticness*, *speechiness*, *instrumentalness*) to stabilize variance before model training.
- Leverage Engineered Features: Retain engineered variables like `duration_minutes`, `popularity_tier`, and `energy_dance_index` as they provide powerful categorical and interactive signals for classifiers.
- Multi-Collinearity Management: Drop or regularize one of the features in strongly correlated pairs (such as *energy* and *loudness*) if using linear models to prevent multi-collinearity issues.

12. **LIMITATIONS OS ANALYSIS**

Dataset & Scope Constraints

- **Genre Representation:** The dataset primarily reflects commercial streaming platforms, potentially underrepresenting ultra-niche, regional, or unrecorded traditional genres.

- **Temporal Limitations:** Audio metrics represent static snapshots of tracks and do not account for how listener preferences, trends, or streaming algorithms evolve over time.

Feature & Measurement Limitations

- **Algorithmic Audio Metrics:** Spotify's audio features (like *danceability*, *valence*, and *energy*) are generated by proprietary machine learning models, which may contain inherent algorithmic biases or oversimplify complex human emotional responses to music.
- **Multicollinearity:** Strong inherent correlations between certain features (such as *energy* and *loudness*) can limit the interpretability of individual feature coefficients in linear models.

Class Imbalance & Sampling

- **Popularity Tier Imbalance:** Chart-topping tracks (*High* tier) represent a smaller fraction (~14%) of the corpus compared to medium and low tiers, which could cause models to be less sensitive to predicting hits accurately.
- **Artist Concentration:** While the top artist represents only ~1.06% of the data, the heavy presence of mainstream global superstars may skew feature distributions toward pop, hip-hop, and mainstream production standards.

14. **FUTURE SCOPE**

Advanced Machine Learning Models: Implement deep learning architectures (such as Recurrent Neural Networks or Convolutional Neural Networks) trained on raw audio spectrograms rather than pre-calculated metadata.

Predictive Hit Forecasting: Build predictive models that estimate a track's future commercial success or popularity tier *before* release based solely on its acoustic profile.

Recommendation System Integration: Integrate the classification and feature similarity pipeline into a functional music recommendation engine or personalized playlist generator.

Temporal Trend Analysis: Analyze how audio feature preferences and genre boundaries have shifted historically over multiple decades using temporal metadata.

Sentiment & Lyric Analysis: Combine audio feature analytics with Natural Language Processing (NLP) on lyric data to analyze the combined impact of words and sound on listener engagement.

15.**CONCLUSION**

- **Comprehensive EDA Success:** Successfully sanitized, processed, and analyzed 7,388 tracks, establishing a clean, robust data foundation free of missing values and formatting inconsistencies.
- **Key Acoustic Insights:** Revealed clear structural patterns, confirming that commercial success (*High* popularity tier) strongly aligns with standardized track lengths (2.5–4 minutes), high energy, and balanced danceability.
- **Mastering & Production Trends:** Identified strong multicollinearity between acoustic drivers like *energy* and *loudness*, underscoring the powerful impact of modern commercial audio compression standards.
- **Dataset Reliability:** Demonstrated high corpus diversity with minimal artist dominance, ensuring that machine learning models will learn generalized, objective acoustic patterns rather than memorizing individual artist profiles.
- **Ready for Modeling:** The comprehensive insights, feature engineering, and outlier management successfully prepare the dataset for high-performance classification and predictive modeling.

In conclusion, this project depicts a comprehensive data-driven analysis of music streaming attributes, revealing the core acoustic and structural patterns that define commercial popularity and genre characteristics. Ultimately, it establishes a robust, clean foundational dataset and provides valuable actionable insights that successfully prepare the ground for advanced machine learning classification and predictive modeling.

16. **REFERENCE**

- Python documentation: https://docs.python.org/3/
- Pandas documentation: https://pandas.pydata.org/docs/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/stable/

- Seaborn documentation: https://seaborn.pydata.org/
- Plotly Python documentation: https://plotly.com/python/

DATSET FROM Test.csv